

Introduction to FPGA Design Project Report

Traffic Light Controller

Prepared By Egehan Özcan 2101925

1. Description of the problem:

Traffic is a big problem in today's era of transportation. cars, buses, taxis and many of the land vehicles that use roads causes land transportation to overfill. That causes a phenomenon called traffic. Traffic causes people to be stuck on the roads for minutes or even hours. Some people don't have time to waste on roads and some people simply don't want to waste their time which is totally justifiable. While reducing traffic can be achieved by multiple means, one is far easier than the others and that one is traffic control and management. Traffic crashes, police control point and many road events can't be controlled or reduced on a big scale, but management of traffic can be easily achieved with proper planning and monitoring

2. Method of solving the given problem:

For solution i developed a traffic light controller system in verilog that can solve the problem. The way how system works is; we have three parameters red, green and yellow which are customizable via changing the parameter name and color code. These three parameters change with respect to state logic that i designed. Each color stays for their designated durations and after the duration ends the state logic checks for the next state and if the next state is available. While these states change's, the output of the code which is "light_o" changes depending on the current and next state variables. This whole system works while the reset function is in its "0" state. If one were to trigger reset with setting its value to one, the state logic resets its color to "RESET_COLOR" which is also customizable and makes the state logic start from scratch. In addition to these features the system also has an enable yellow function which can bypass the default order of lights (red,yellow,green) when set to zero and makes it so that the lights cycle between red and green states. All the features stated in here can be seen on codes

3. Verilog code of solution:

Main code:

```
module TrafficLightController(
input clk_i,
input rst_i,
input ENABLE_YELLOW,
input [31:0] RED_DURATION,
input [31:0] YELLOW_DURATION,
input [31:0] GREEN_DURATION,
input [2:0] RESET_COLOR,
output reg [2:0] light_o
);

reg [31:0] counter;
reg [2:0] next_state; // Added next state for better timing
reg [2:0] state; // Current state

// Define states
localparam RED = 3'b001, YELLOW = 3'b010, GREEN = 3'b100;
// Next state and output logic
always @(*) begin

    next_state = state; // Default: keep current state

    case (state)
        RED: begin
            if (counter >= (RED_DURATION - 1)) begin
                next_state = ENABLE_YELLOW ? YELLOW : GREEN;
            end
        end
        YELLOW: begin

            if (counter >= (YELLOW_DURATION - 1)) begin
                next_state = GREEN;
            end
        end
    endcase
end
```

```

end

GREEN: begin
    if (counter >= (GREEN_DURATION - 1)) begin
        next_state = RED;
    end
end

default: next_state = RED;
endcase
end

// State and counter update
always @(posedge clk_i) begin
    if (rst_i) begin
        state <= RESET_COLOR;
        counter <= 0;
        light_o <= RESET_COLOR;
    end else begin
        // Update state
        state <= next_state;
        light_o <= next_state;
        // Update counter
        if (state != next_state) begin

            counter <= 0; // Reset counter on state change
        end else begin
            counter <= counter + 1;
        end
    end
end
end
endmodule

```

4. Verilog code of testbenches

Clock and Sequence Test:

```
module TrafficLightController_tb; // Inputs and outputs reg clk_i, rst_i,
ENABLE_YELLOW; reg [31:0] RED_DURATION, YELLOW_DURATION,
GREEN_DURATION; reg [2:0] RESET_COLOR; wire [2:0] light_o;

// Define light colors for better readability
localparam RED = 3'b001;
localparam YELLOW = 3'b010;
localparam GREEN = 3'b100;
// Counter to track how long each light stays on
reg [31:0] state_duration;
// Connect the traffic light controller
TrafficLightController tlc (

    .clk_i(clk_i),
    .rst_i(rst_i),
    .ENABLE_YELLOW(ENABLE_YELLOW),
    .RED_DURATION(RED_DURATION),
    .YELLOW_DURATION(YELLOW_DURATION),
    .GREEN_DURATION(GREEN_DURATION),
    .RESET_COLOR(RESET_COLOR),
    .light_o(light_o)
);

// Generate clock: toggles every 5 time units
initial begin
    clk_i = 0;
    forever #5 clk_i = ~clk_i;
end

// Task to verify if a light stays on for the correct duration
task check_light_timing;
    input [2:0] expected_light;
    input [31:0] expected_duration;
    begin
        // Wait until the expected light turns on
        wait(light_o == expected_light);

        // Count how long the light stays on
```

```

state_duration = 0;
while (state_duration < expected_duration && light_o === expected_light)
begin
    @(posedge clk_i);
    state_duration = state_duration + 1;
end

// Check if light changed too early
if (state_duration < expected_duration) begin
    case(expected_light)
        RED: $display("ERROR: RED light changed too early at time %t",
$time);
        YELLOW: $display("ERROR: YELLOW light changed too early at
time %t", $time);
        GREEN: $display("ERROR: GREEN light changed too early at time
%t", $time);
    endcase
    $display("Expected duration: %d, Actual duration: %d",
expected_duration, state_duration);
end
// Check if light stayed on too long
else begin
    @(posedge clk_i);
    if (light_o === expected_light) begin
        case(expected_light)
            RED: $display("ERROR: RED light stayed on too long at time
%t", $time);
            YELLOW: $display("ERROR: YELLOW light stayed on too long
at time %t", $time);
            GREEN: $display("ERROR: GREEN light stayed on too long at
time %t", $time);
        endcase
    end else begin
        case(expected_light)
            RED: $display("PASS: RED light timing correct (Duration:
%d)", expected_duration);
            YELLOW: $display("PASS: YELLOW light timing correct
(Duration: %d)", expected_duration);

```

```

        GREEN: $display("PASS: GREEN light timing correct (Duration:
%d)", expected_duration);
        endcase
    end
end
endtask

// Main test sequence
initial begin
    // Test 1: Normal operation with all lights
    $display("\nTest 1: Testing normal operation (all lights)");
    // Initialize values
    rst_i = 1;
    RESET_COLOR = RED;
    RED_DURATION = 10;
    YELLOW_DURATION = 5;
    GREEN_DURATION = 8;
    ENABLE_YELLOW = 1;

    // Start test
    #10 rst_i = 0; // Release reset
    // Check timing for each light (2 complete cycles)
    repeat(2) begin

        check_light_timing(RED, RED_DURATION);
        check_light_timing(YELLOW, YELLOW_DURATION);
        check_light_timing(GREEN, GREEN_DURATION);
    end
    // Test 2: Operation without yellow light
    $display("\nTest 2: Testing operation without yellow light");
    ENABLE_YELLOW = 0;
    repeat(2) begin

        check_light_timing(RED, RED_DURATION);
        check_light_timing(GREEN, GREEN_DURATION);
    end

    // Test 3: Different timing values

```

```

$display("\nTest 3: Testing with different durations");
ENABLE_YELLOW = 1;
RED_DURATION = 15;
YELLOW_DURATION = 3;
GREEN_DURATION = 12;

repeat(2) begin

    check_light_timing(RED, RED_DURATION);
    check_light_timing(YELLOW, YELLOW_DURATION);
    check_light_timing(GREEN, GREEN_DURATION);
end

#100 $finish;

end

// Intermediate wires for monitoring
wire is_red = (light_o == RED);
wire is_yellow = (light_o == YELLOW);
wire is_green = (light_o == GREEN);
// Monitor changes
initial begin

    $monitor("Time=%t, light_o=%b (RED=%b, YELLOW=%b,
GREEN=%b)",
        $time, light_o, is_red, is_yellow, is_green);
end

// Create waveform file for viewing in waveform viewer
initial begin

    $dumpfile("traffic_light.vcd");
    $dumpvars(0, TrafficLightController_tb);
end
endmodule

```

Enable Yellow Functionality Test:

```
module TrafficLightController_tb;

reg clk_i,

rst_i,

ENABLE_YELLOW;

reg [31:0] RED_DURATION, YELLOW_DURATION, GREEN_DURATION;

reg [2:0] RESET_COLOR;

wire [2:0] light_o;

// Instantiate the Traffic Light Controller
TrafficLightController tlc (
    .clk_i(clk_i),
    .rst_i(rst_i),
    .ENABLE_YELLOW(ENABLE_YELLOW),
    .RED_DURATION(RED_DURATION),
    .YELLOW_DURATION(YELLOW_DURATION),
    .GREEN_DURATION(GREEN_DURATION),
    .RESET_COLOR(RESET_COLOR),
    .light_o(light_o)
);

// Clock generation

initial begin
    clk_i = 0;
    forever #5 clk_i = ~clk_i; // Toggle clock every 5 time units
end

// Test scenarios

initial begin

    // Initialize VCD dump file
    $dumpfile("traffic_light.vcd");
    $dumpvars(0, TrafficLightController_tb);

    // Initialize parameters
    rst_i = 1;
    RED_DURATION = 5;
    YELLOW_DURATION = 3;
    GREEN_DURATION = 7;
    ENABLE_YELLOW = 1;

    // Use the binary values for RESET_COLOR (3'b001 for RED, 3'b010 for YELLOW,
```



```

3'b100 for GREEN)
  RESET_COLOR = 3'b001; // Set initial state to RED

  #10 rst_i = 0; // Release reset

  // Test with YELLOW enabled
  #900 $display("ENABLE_YELLOW = 0");
  ENABLE_YELLOW = 0; // Test without YELLOW

  #900;
  $dumpflush; // Flush the VCD file
  $finish; // End simulation (using $finish instead of $stop for better tool compatibility)
end
// Monitoring changes
initial begin

  $monitor("At time %t, light_o = %b", $time, light_o);
end

endmodule

```

Reset Functionality Test:

```
module TrafficLightController_tb;

reg clk_i, rst_i, ENABLE_YELLOW;

reg [31:0] RED_DURATION, YELLOW_DURATION, GREEN_DURATION;

reg [2:0] RESET_COLOR;

wire [2:0] light_o;

// Parameters for readability
localparam RED = 3'b001;
localparam YELLOW = 3'b010;
localparam GREEN = 3'b100;
// Instantiate the Traffic Light Controller
TrafficLightController tlc (

    .clk_i(clk_i),
    .rst_i(rst_i),
    .ENABLE_YELLOW(ENABLE_YELLOW),
    .RED_DURATION(RED_DURATION),
    .YELLOW_DURATION(YELLOW_DURATION),
    .GREEN_DURATION(GREEN_DURATION),
    .RESET_COLOR(RESET_COLOR),
    .light_o(light_o)
);
// Clock generation
initial begin
    clk_i = 0;
    forever #5 clk_i = ~clk_i;
end
// Task to check reset behavior
task check_reset;

    input [2:0] expected_color;
    begin
        rst_i = 1;
        #10; // Wait for one clock cycle
        if (light_o !== expected_color) begin
            $display("Reset Test Failed! Expected: %b, Got: %b", expected_color, light_o);
        end else begin
```

```

        $display("Reset Test Passed! Color matches expected: %b", light_o);
    end
    rst_i = 0;
    #10; // Wait for system to stabilize
end
endtask

// Test scenarios
initial begin

    // Test case 1: Reset to RED
    $display("\nTest Case 1: Reset to RED");
    RESET_COLOR = RED;
    RED_DURATION = 5;
    YELLOW_DURATION = 3;
    GREEN_DURATION = 7;
    ENABLE_YELLOW = 1;
    check_reset(RED);
    // Test case 2: Reset to YELLOW
    $display("\nTest Case 2: Reset to YELLOW");
    RESET_COLOR = YELLOW;
    check_reset(YELLOW);
    // Test case 3: Reset to GREEN
    $display("\nTest Case 3: Reset to GREEN");
    RESET_COLOR = GREEN;
    check_reset(GREEN);
    // Test case 4: Reset during operation
    $display("\nTest Case 4: Reset during operation");
    rst_i = 0;
    RESET_COLOR = RED;
    #50; // Let the system run for a while
    check_reset(RED);
    // Test case 5: Multiple rapid resets
    $display("\nTest Case 5: Multiple rapid resets");
    repeat(3) begin

        check_reset(RED);
        #3;
    end
    #100 $finish;

end

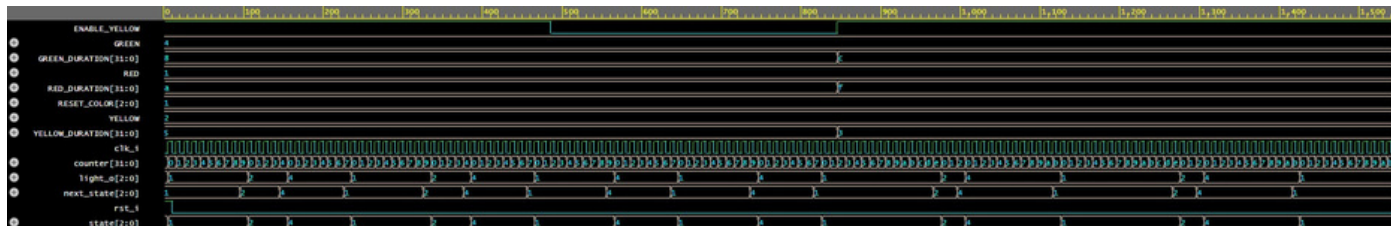
```

```
// Monitor changes
initial begin
    $monitor("Time=%t rst_i=%b RESET_COLOR=%b light_o=%b",
        $time, rst_i, RESET_COLOR, light_o);
end

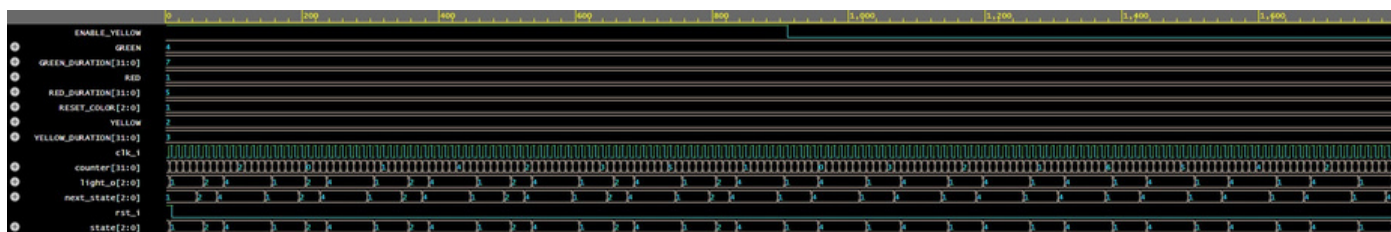
// Generate VCD file for waveform viewing
initial begin
    $dumpfile("traffic_light.vcd");
    $dumpvars(0, TrafficLightController_tb);
end
endmodule
```

5. Images (screenshots) of the waveform output of simulation

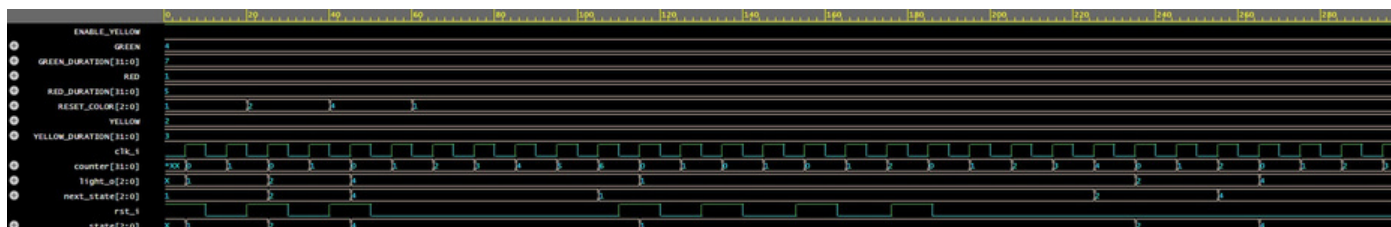
Clock and Sequence Test:



Enable Yellow Test:



Reset Functionality Test:



6. Code outputs:

Clock and Sequence Test:

Test 1: Testing normal operation (all lights) VCD info: dumpfile traffic_light.vcd opened for output. Time= 0, light_o=xxx (RED=x, YELLOW=x, GREEN=x) Time= 5, light_o=001 (RED=1, YELLOW=0, GREEN=0) Time= 105, light_o=010 (RED=0, YELLOW=1, GREEN=0) PASS: RED light timing correct (Duration: 10) Time= 155, light_o=100 (RED=0, YELLOW=0, GREEN=1)

PASS: YELLOW light timing correct (Duration: 5) Time= 235, light_o=001 (RED=1, YELLOW=0, GREEN=0) ERROR: GREEN light changed too early at time 245 Expected duration: 8, Actual duration: 7 Time= 335, light_o=010 (RED=0, YELLOW=1, GREEN=0) PASS: RED light timing correct (Duration: 10) Time= 385, light_o=100 (RED=0, YELLOW=0, GREEN=1) ERROR: YELLOW light changed too early at time 395 Expected duration: 5, Actual duration: 4 Time= 465, light_o=001 (RED=1, YELLOW=0, GREEN=0) PASS: GREEN light timing correct (Duration: 8)

Test 2: Testing operation without yellow light

Time= 565, light_o=100 (RED=0, YELLOW=0, GREEN=1) ERROR: RED light changed too early at time 575 Expected duration: 10, Actual duration: 9 Time= 645, light_o=001 (RED=1, YELLOW=0, GREEN=0) PASS: GREEN light timing correct (Duration: 8) Time= 745, light_o=100 (RED=0, YELLOW=0, GREEN=1) ERROR: RED light changed too early at time 755 Expected duration: 10, Actual duration: 9 Time= 825, light_o=001 (RED=1, YELLOW=0, GREEN=0) PASS: GREEN light timing correct (Duration: 8)

Test 3: Testing with different durations

Time= 975, light_o=010 (RED=0, YELLOW=1, GREEN=0) ERROR: RED light changed too early at time 985 Expected duration: 15, Actual duration: 14 Time= 1005, light_o=100 (RED=0, YELLOW=0, GREEN=1) PASS: YELLOW light timing correct (Duration: 3) Time= 1125, light_o=001 (RED=1, YELLOW=0, GREEN=0) ERROR: GREEN light changed too early at time 1135 Expected duration: 12, Actual duration: 11 Time= 1275, light_o=010 (RED=0, YELLOW=1, GREEN=0) PASS: RED light timing correct (Duration: 15) Time= 1305, light_o=100 (RED=0, YELLOW=0, GREEN=1) ERROR: YELLOW light changed too early at time 1315 Expected duration: 3, Actual duration: 2 Time= 1425, light_o=001 (RED=1, YELLOW=0, GREEN=0) PASS: GREEN light timing correct (Duration: 12) testbench.sv:122: \$finish called at 1545 (1s) Finding VCD file...

./traffic_light.vcd

Enable Yellow Test:

```
VCD info: dumpfile traffic_light.vcd opened for output. At
time 0, light_o = xxx At time 5, light_o = 001 At time 55,
light_o = 010 At time 85, light_o = 100 At time 155, light_o =
001 At time 205, light_o = 010 At time 235, light_o = 100 At
time 305, light_o = 001
At time 355, light_o = 010 At time 385, light_o =
100 At time 455, light_o = 001 At time 505, light_o
= 010 At time 535, light_o = 100 At time 605,
light_o = 001 At time 655, light_o = 010 At time
685, light_o = 100 At time 755, light_o = 001 At
time 805, light_o = 010 At time 835, light_o = 100
At time 905, light_o = 001
ENABLE_YELLOW = 0
At time 955, light_o = 100
At time 1025, light_o = 001
At time 1075, light_o = 100
At time 1145, light_o = 001 At time 1195, light_o =
100 At time 1265, light_o = 001 At time 1315,
light_o = 100 At time 1385, light_o = 001 At time
1435, light_o = 100 At time 1505, light_o = 001 At
time 1555, light_o = 100 At time 1625, light_o = 001
At time 1675, light_o = 100
At time 1745, light_o = 001
At time 1795, light_o = 100
testbench.sv:49: $finish called at 1810 (1s)
```

Reset Functionality Test:

Test Case 1: Reset to RED VCD info: dumpfile traffic_light.vcd opened for output. Time= 0 rst_i=1 RESET_COLOR=001 light_o=xxx Time= 5 rst_i=1 RESET_COLOR=001 light_o=001 Reset Test Passed! Color matches expected: 001 Time= 10 rst_i=0 RESET_COLOR=001 light_o=001

Test Case 2: Reset to YELLOW

Time= 20 rst_i=1 RESET_COLOR=010 light_o=001
Time= 25 rst_i=1 RESET_COLOR=010 light_o=010
Reset Test Passed! Color matches expected: 010
Time= 30 rst_i=0 RESET_COLOR=010 light_o=010

Test Case 3: Reset to GREEN Time= 40 rst_i=1
RESET_COLOR=100 light_o=010 Time= 45 rst_i=1
RESET_COLOR=100 light_o=100 Reset Test Passed!
Color matches expected: 100 Time= 50 rst_i=0
RESET_COLOR=100 light_o=100

Test Case 4: Reset during operation

Time= 60 rst_i=0 RESET_COLOR=001 light_o=100
Time= 110 rst_i=1 RESET_COLOR=001 light_o=100
Time= 115 rst_i=1 RESET_COLOR=001 light_o=001
Reset Test Passed! Color matches expected: 001
Time= 120 rst_i=0 RESET_COLOR=001 light_o=001

Test Case 5: Multiple rapid resets

Time= 130 rst_i=1 RESET_COLOR=001 light_o=001
Reset Test Passed! Color matches expected: 001
Time= 140 rst_i=0 RESET_COLOR=001 light_o=001
Time= 153 rst_i=1 RESET_COLOR=001 light_o=001
Reset Test Passed! Color matches expected: 001
Time= 163 rst_i=0 RESET_COLOR=001 light_o=001
Time= 176 rst_i=1 RESET_COLOR=001 light_o=001
Reset Test Passed! Color matches expected: 001
Time= 186 rst_i=0 RESET_COLOR=001 light_o=001
Time= 235 rst_i=0 RESET_COLOR=001 light_o=010
Time= 265 rst_i=0 RESET_COLOR=001 light_o=100
testbench.sv:81: \$finish called at 299 (1s)